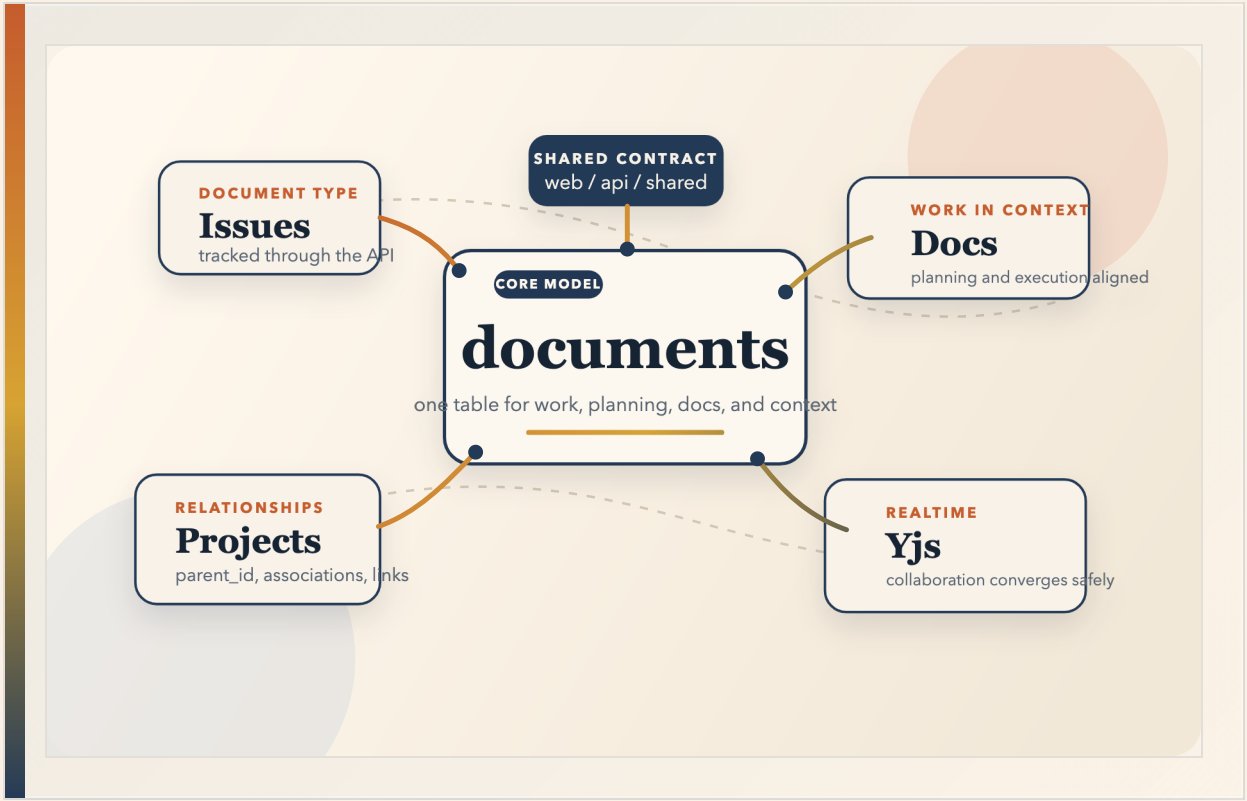
SHIPSHAPE DEMO



ShipShape

Architecture, audit findings, and a practical improvement strategy for a unified work-and-collaboration system.

AUTHOR
Stefano Caruso
AI Product Engineer

CORE THESIS

Strong architecture, uneven maturity

FOCUS

Understand first, then improve

OUTCOME

Practical path forward

APPROACH

I started by building a mental model, not by changing code

What I focused on

- Package boundaries across `web/`, `api/`, and `shared/`
- Request flow from UI to API to database
- Realtime collaboration flow over WebSocket and Yjs
- How the document model shapes the whole product

Why that matters

- It reduces the risk of superficial fixes
- It makes audit findings easier to explain and prioritize
- It turns code reading into an architecture-level assessment

The first deliverable was understanding the system safely enough to change it.

ARCHITECTURE

ShipShape is a unified-document monolith with clear package boundaries

`web/`

- React frontend
- REST for normal app data
- WebSocket for realtime collaboration

→

`shared/`

- Types
- Enums
- Constants
- Helpers

→

`api/`

- Express + TypeScript backend
- Validation and persistence
- Realtime auth and Yjs coordination

The split is strong because both sides reuse the same contract layer instead of drifting apart.

DATA MODEL

The unified document model is the most important design decision

Instead of separate main tables for issues, projects, docs, and sprints, Ship stores them in one central `documents` table.

DOCUMENT TYPE

Filters the central table to the subset each query needs.

HIERARCHY

`parent_id` models structural nesting.

RELATIONSHIPS

`document_associations` and `document_links` layer meaning on top.

This model gives Ship flexibility and product coherence, but it also increases the cost of graph-heavy access patterns.

REQUEST FLOW

Create Issue is a clean example of the runtime model

01 User clicks

Create Issue

02 Frontend

sends a `POST` request

03 Backend

validates the payload

04 Backend

inserts a row into `documents`

05 API returns

JSON

06 Frontend re-

renders from the response

The frontend never writes directly to the database. The API remains the control point.

REALTIME

Collaboration is one of the clearest differentiators in the system

Flow

- The editor opens a document-specific WebSocket room
- The backend authenticates using the existing session cookie
- The connection upgrades into a collaboration channel
- Yjs syncs concurrent edits and the server persists state to Postgres

Why it is strong

- CRDT merging lets simultaneous edits converge automatically
- Docs, planning, and execution stay in one system
- This is a real product advantage, not just a technical feature

ASSESSMENT

The architecture has a strong core, but the risks are concentrated in consistency

Strongest decisions

- Unified document model
- Clear `web` / `api` / `shared` split
- Yjs-based collaboration model

Weakest points

- Relationship consistency is too fragile
- Collaborative state has too many sources of truth
- `shared/` is valuable, but not yet fully authoritative

The big gap is not architectural vision. It is operational consistency across layers.

AUDIT

My findings were mixed: strong design choices, clear implementation risks

AREA	MAIN FINDING	IMPLICATION
Type safety	Strict mode is on, but API routes rely heavily on casts	Type guarantees are weaker than they appear
Frontend performance	The initial bundle is still too large	Slower first load and heavier critical path
Database and API	Graph-heavy flows raise query cost and are likely bottlenecks	Scalability risk grows with data volume and concurrency
Testing	Coverage exists, but reliability and full-suite completion are weak	The safety net is not mature enough

PERFORMANCE

The clearest frontend opportunity is reducing the amount of code shipped up front

WHAT IS WORKING

Code splitting already exists.

WHAT IS NOT

The main app chunk is still oversized.

WHY

Heavy dependencies are still landing in the initial path.

BEST NEXT MOVE

Reduce what the browser has to download before the app becomes useful.

SCALABILITY

The unified document model works, but relationship-heavy flows are where the cost starts to rise

Database finding

Query fan-out grows in relationship-heavy flows, especially when the system has to traverse the graph around the central `documents` table.

API finding

Latency is acceptable on the sampled endpoints, but graph-heavy endpoints are the ones most likely to become bottlenecks as scale increases.

The optimization target is not the entire stack equally. It is the relationship access patterns around the central model.

QUALITY

Testing, runtime behavior, and accessibility still need operational maturity

TESTING

Meaningful coverage exists, but reliability and critical-flow confidence are still not strong enough.

RUNTIME BEHAVIOR

The issue is not only whether errors happen, but whether users are informed and can recover safely.

ACCESSIBILITY

The baseline still shows 5 serious violations, partial keyboard completeness, 28 contrast failures, and ARIA gaps.

BOTTOM LINE

Good architecture does not matter much if quality signals are not dependable in production.

POSITIONING

Ship's advantage is unified work and context, not breadth of PM features

Today

- The LLM layer exists, but it is narrow and optional
- It mainly supports advisory analysis plus Claude context APIs
- Availability depends on Bedrock credentials

Competitive framing

- Jira is stronger as a mature PM operating system
- Ship is stronger when docs, planning, execution, and collaboration stay tightly connected
- That advantage only holds if reliability and consistency catch up to the vision

STRATEGY

The improvement strategy should follow the architecture's actual pressure points

STEP 1

Consolidate relationship handling

Reduce inconsistency around how documents relate to each other.

STEP 2

Strengthen the shared contract layer

Make `shared/` more authoritative so the frontend and backend cannot drift.

STEP 3

Harden realtime and scaling

Reduce multi-source state risk and improve behavior under concurrency and growth.

STEP 4

Improve test reliability and accessibility

Increase confidence in changes and improve real usability.

This order fixes structural consistency first, then correctness, then scale, then safety and usability.

CONCLUSION

Ship has a differentiated core. The next step is operational maturity.

Ship does not need a new architecture first. It needs to fully realize the one it already has.

- Strong architectural core
- Clear product differentiation
- Main gap: consistency and reliability
- Best path: targeted hardening, not reinvention

Q&A

Questions on architecture, audit findings, or the improvement strategy.